

16차시	총10문제		연습: ☐	과제 : ☒	평가 : ☐
1. 스택 자료구조의 주요 특징은 무엇인가요? a) FIFO (First In First Out) b) LIFO (Last In First Out) c) 정렬된 데이터 관리 d) 랜덤 접근 가능 답: b 2. 파이썬 리스트를 스택으로 사용할 때 요소를 제거하는 메서드는 무엇인가요? a) pop() b) remove() c) del d) discard() 답: a 3. 큐 자료구조의 주요 특징은 무엇인가요? a) LIFO (Last In First Out) b) FIFO (First In First Out) c) 정렬된 데이터 관리 d) 랜덤 접근 가능 답: b					

4. 덱(Deque)의 주요 특징은 무엇인가요?

- a) 한쪽에서만 삽입과 삭제가 가능하다.
- b) 양쪽 끝에서 삽입과 삭제가 가능하다.
- c) 데이터를 자동으로 정렬한다.
- d) LIFO만 지원한다.

답: b

5. 다음 자료 구조의 활용 예시 중 구현 방식이 다른 하나는 무엇인가요?

- a) 함수 호출 스택
- b) 프로세스 관리 (예: 작업 대기열)
- c) 괄호 검사와 같은 문자열 처리
- d) 웹 브라우저 방문기록 (뒤로가기)

답: b

6. 리스트를 이용하여 기본적인 스택(stack)을 구현하세요. (다음 내용을 고려할 것.)

push(x): 정수 x를 스택에 삽입

pop(): 스택에서 가장 위의 값을 제거하고 반환. 만약 스택이 비어 있다면 -1을 반환

top(): 스택의 가장 위에 있는 값을 반환. 만약 스택이 비어 있다면 -1을 반환

is_empty(): 스택이 비어 있으면 True, 아니면 False를 반환

답:

```
class Stack():
    def __init__(self):
        self.stack = []

    def push(self,data):
        self.stack.append(data)

    def pop(self):
        if not self.is_empty():
            return self.stack.pop()
        return -1

    def is_empty(self):
        if len(self.stack) == 0:
            return True
        return False

    def top(self):
        if not self.is_empty():
            return self.stack[-1]
        return -1

    def status_stack(self):
        return self.stack
```

7. 후위 표기법(Postfix Notation, Reverse Polish Notation)으로 주어진 수식을 계산하는 프로그램을 작성하세요.

후위 표기법에서는 연산자가 피연산자 뒤에 위치합니다. 예를 들어, 3 4 +는 3 + 4를 의미하며, 결과는 7입니다.

연산자는 +, -, *, /만 고려합니다. (복수 연산자도 처리 가능해야 함.)

답:

```
def postfix(expression):  
    stack = []  
    tokens = expression.split()  
  
    for token in tokens:  
        if token.isdigit():  
            stack.append(int(token))  
        else:  
            b = stack.pop()  
            a = stack.pop()  
            if token == '+':  
                stack.append(a + b)  
            elif token == '-':  
                stack.append(a - b)  
            elif token == '*':  
                stack.append(a * b)  
            elif token == '/':  
                stack.append(a / b)  
  
    return stack.pop()
```

8. 리스트를 사용하여 기본적인 큐(queue)를 구현하세요. (다음 내용을 고려할 것.)

enqueue(x): 정수 x를 큐의 끝에 추가

dequeue(): 큐의 앞에서 값을 제거하고 반환. 만약 큐가 비어 있다면 -1을 반환

front(): 큐의 앞에 있는 값을 반환. 만약 큐가 비어 있다면 -1을 반환

is_empty(): 큐가 비어 있으면 True, 아니면 False를 반환

답:

```
class Queue:

    def __init__(self):
        self.queue = []

    def enqueue(self,data):
        self.queue.append(data)

    def dequeue(self):
        if not self.is_empty():
            return self.queue.pop(0)
        return -1

    def is_empty(self):
        if len(self.queue) == 0:
            return True
        return False

    def front(self):
        if not self.is_empty():
            return self.queue.pop[0]
        return -1
```

9. 고정된 크기의 원형 큐를 구현하세요. (다음 내용을 고려할 것.)

enqueue(x): 정수 x를 큐의 끝에 추가. (큐가 가득 차면 추가하지 않음)

dequeue(): 큐의 앞에서 값을 제거하고 반환. (큐가 비어 있다면 -1 반환)

front(): 큐의 앞에 있는 값을 반환. (비어 있다면 -1 반환)

is_empty(): 큐가 비어 있으면 True, 아니면 False 반환

is_full(): 큐가 가득 차 있으면 True, 아니면 False 반환

답:

```
class CircularQueue:
    def __init__(self, max_size):
        self.max_size = max_size
        self.queue = [None] * max_size
        self.front_idx = 0
        self.rear_idx = 0
        self.count = 0

    def enqueue(self, data):
        if not self.is_full():
            self.queue[self.rear_idx] = data
            self.rear_idx = (self.rear_idx + 1) % self.max_size
            self.count += 1
        else:
            return

    def dequeue(self):
        if not self.is_empty():
            value = self.queue[self.front_idx]
            self.queue[self.front_idx] = None
            self.front_idx = (self.front_idx + 1) % self.max_size
            self.count -= 1
            return value
        return -1

    def front(self):
```

```
        if not self.is_empty():
            return self.queue[self.front_idx]
        return -1

    def is_empty(self):
        return self.count == 0

    def is_full(self):
        return self.count == self.max_size

    def status_queue(self):
        return self.queue

    def rotate_queue(self, k):
        for _ in range(k):
            value = self.dequeue()
            if value != -1:
                self.enqueue(value)
        return self.status_queue()
```

10. 리스트를 이용하여 덱(deque, 양방향 큐)을 구현하세요. (다음 내용을 고려할 것.)

push_front(x): 정수 x를 덱의 앞에 추가

push_back(x): 정수 x를 덱의 뒤에 추가

pop_front(): 덱의 앞에서 값을 제거하고 반환 (비어 있다면 -1)

pop_back(): 덱의 뒤에서 값을 제거하고 반환 (비어 있다면 -1)

답:

```
class Deque():

    def __init__(self):
        self.queue = []

    def push_front(self,data):
        self.queue.insert(0,data)

    def push_back(self,data):
        self.queue.append(data)

    def pop_front(self):
        if not self.is_empty():
            return self.queue.pop(0)
        return -1

    def pop_back(self):
        if not self.is_empty():
            return self.queue.pop()
        return -1

    def is_empty(self):
        if len(self.queue) == 0:
            return True
        return False

    def status_queue(self):
        return self.queue
```